SALUS SECURITY

OCT 2026



CODE SECURITY ASSESSMENT

VISTARA TOKEN

Overview

Project Summary

- Name: Vistara Token
- Platform: EVM-compatible chains
- Language: Solidity
- Address: 0xf16b98371CB191F392b87223fb143d0a8116aE91
- Audit Range: See Appendix - 1

Project Dashboard

Application Summary

Name	Vistara Token
Version	v2
Type	Solidity
Dates	Oct 19 2026
Logs	Oct 19 2026; Oct 19 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	0
Total Low-Severity issues	0
Total informational issues	2
Total	2

Contact

E-mail: support@salusec.io

SALUS

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

SALUS

Content

Introduction **4**
1.1 About SALUS 4
1.2 Audit Breakdown 4
1.3 Disclaimer 4
Findings **5**
2.1 Summary of Findings 5
2.2 Notable Findings 6
2.3 Informational Findings 7
1. Use of floating pragma 7
2. Lack of Security Contact Information for Responsible Disclosure 8
Appendix **9**
Appendix 1 - Files in Scope 9

SALUS

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

SALUS

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Use of floating pragma	Informational	Configuration	Acknowledged
2	Lack of Security Contact Information for Responsible Disclosure	Informational	Configuration	Acknowledged

SALUS

2.2 Notable Findings

No notable issues were found.

SALUS

2.3 Informational Findings

1. Use of floating pragma

Severity: Informational	Category: Configuration
Target: - VistaraToken.sol	

Description

```
pragma solidity ^0.8.27;
```

For example, the `VistaraToken` uses a floating compiler version ^0.8.27.

Using a floating pragma ^0.8.27 statement is discouraged, as code may compile to different bytecodes with different compiler versions. Use a locked pragma statement to get a deterministic bytecode. Also use the latest Solidity version to get all the compiler features, bug fixes and optimizations.

Recommendation

It is recommended to use a locked Solidity version throughout the project. It is also recommended to use the most stable and up-to-date version.

Status

This issue has been acknowledged by the team.

SALUS

2. Lack of Security Contact Information for Responsible Disclosure

Severity: Informational	Category: Configuration
Target: - VistaraToken.sol	

Description

The contract `VistaraToken` does not specify a security contact point. Including a designated security contact (such as an email address or ENS name) in the contract's NatSpec header facilitates responsible vulnerability disclosure. This makes it easier for external researchers to quickly reach the appropriate team in the event a vulnerability is identified, helping minimize the time window between discovery and mitigation. The Ethereum community has begun standardizing this practice using the `@custom:security-contact` tag, adopted by tools such as OpenZeppelin Wizard and ethereum-lists.

Recommendation

Consider adding a NatSpec comment at the top of the contract with a `@custom:security-contact` field pointing to the preferred disclosure channel.

Status

This issue has been acknowledged by the team.

SALUS

Appendix

Appendix 1 - Files in Scope

This audit covered the following deployed contract at 0xf16b98371CB191F392b87223fb143d0a8116aE91 on Base Chain:

File	SHA-1 hash
VistaraToken.sol	537640eafae8bd87faa2309173628a9c4da98f86

SALUS